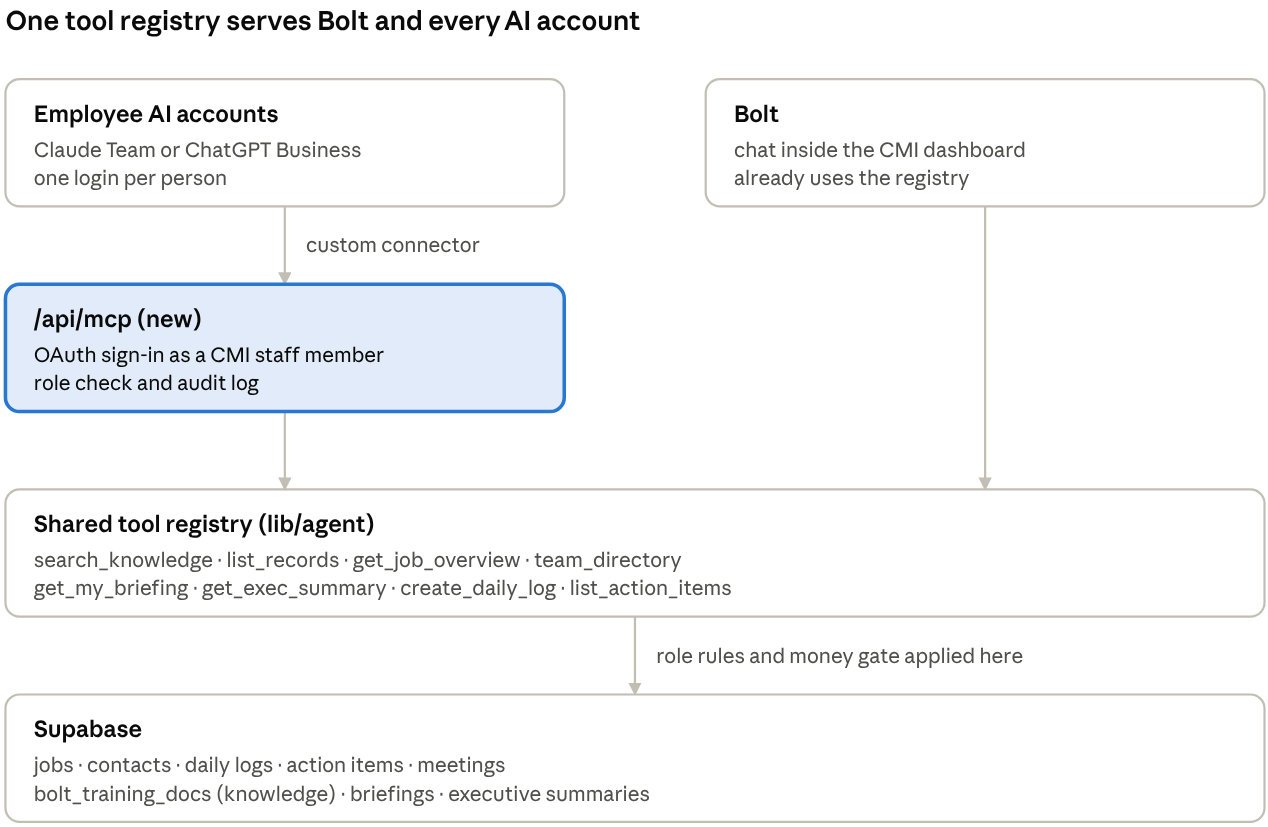
CMI MCP Server — Game Plan

Oct 21, 2026 · @Jeremy Waters

Where It Fits

The MCP server is a thin new door into tools Bolt already uses, so every AI account gets the same data and permissions as the dashboard.



CMI MCP server · where it sits in the web app

Only `/api/mcp` is new; the registry and tables already exist in Qallus/cmi.

What Already Exists in Qallus/cmi

Most of the MCP server already exists as Bolt's tool system; the job is to expose it, not build it. The app lives in `apps/cmi-next` (Next.js 16, React 19, Supabase, deployed on Coolify via Docker).

Existing piece	Where	Reuse for MCP
Bolt tool registry (13 tools: list/get/create/update/delete records, job overview, team directory, dashboard help, send message, projections)	`lib/agent/tools.ts`, `lib/agent/registry.ts`	Becomes the MCP tool list
28 entities (jobs, contacts, daily logs, action items, meetings, invoices, documents…)	`lib/agent/entities.ts`	Records the tools can read and write
Role access + money gate (`BOLT_ROLES`, `BOLT_FINANCIAL_ROLES`)	`lib/agent/access.ts`, `guard.ts`	Same permissions apply in Claude/ChatGPT
Confirm-before-send/delete (`PendingAction`)	`lib/agent/types.ts`	Keep for risky tools
Knowledge base (`bolt_training_docs`)	`lib/agent/training.ts`	Seed of the company knowledge base
Per-person morning briefing	`lib/briefing/build.ts`	`get_my_briefing` tool
Executive summary	`app/api/exec-summary`	`get_exec_summary` tool
Staff sessions and roles	`lib/auth`	Maps a connected user to a staff role

Gaps: no MCP endpoint or MCP SDK yet, the knowledge base is injected whole into the prompt (capped at 24,000 characters) instead of searched, and action items are job tasks, not commitments pulled from email.

Design Decisions

Build it as one route inside the existing app, sharing Bolt's code, so Bolt and the AI accounts always see the same tools and permissions.

Decision	Recommendation	Why
Where it runs	A route in `apps/cmi-next`: `my.constructedmatter.com/api/mcp`	Same deploy, same Supabase client, no new server
Protocol	MCP Streamable HTTP via the official `@modelcontextprotocol/sdk`	What Claude and ChatGPT custom connectors connect to
Login	OAuth sign-in with the user's CMI staff account	Each call knows who is asking, so role rules apply
Permissions	Reuse `canUseBolt`, `canReadFinancials`, per-entity write roles	One permission system, not two
Risky actions	Read-only first; sends and deletes stay off or require confirmation	AI accounts call tools without Bolt's confirm screen
Shared code	Move tool definitions into one registry both Bolt and MCP read	A tool added once appears in both
Audit	Log every MCP call (who, tool, inputs, result)	Visibility and troubleshooting

First Tools to Ship

Ship eight tools in version 1, mostly wrappers on code that already runs; only `search_knowledge` needs a schema change.

Tool	Does	Built from	New schema
`search_knowledge`	Finds SOPs, templates, standards	`bolt_training_docs`	Add category, access level, version, text search index
`list_records` / `get_record`	Reads jobs, contacts, daily logs, meetings, etc.	Bolt registry	None

Tool	Does	Built from	New schema
`get_job_overview`	One job's status, team, open items	Bolt `get_job_overview`	None
`get_my_briefing`	The caller's morning briefing	`lib/briefing/build.ts`	None
`get_exec_summary`	Company-wide summary (leadership only)	`exec-summary`	None
`team_directory`	Who does what	Bolt `team_directory`	None
`create_daily_log`	Saves a formatted log to a job	Bolt `create_record` (daily_log)	None
`list_action_items`	Open items by job or person	`action_items`	None

Later: `get_commitments` and `update_commitment` once the commitments table and email ingestion exist, plus an `mcp_audit_log` table in version 1.

Build Steps

About 6–8 working days to a version the team can use; sign-in (step 3) is the riskiest step.

1. **Scaffold** — add `@modelcontextprotocol/sdk`, create `app/api/mcp/route.ts`, test locally with MCP Inspector (0.5 day)
2. **Share the registry** — pull Bolt's tool definitions into one module both Bolt and MCP read; expose read-only tools (1 day)
3. **Sign-in** — OAuth with CMI staff accounts; map the user to `StaffContext` so role rules apply (1.5–2 days)
4. **Knowledge search** — migration adding category, access level, version, and a text search index to `bolt_training_docs`; build `search_knowledge` (1 day)
5. **Briefing tools** — `get_my_briefing`, `get_exec_summary` (leadership only) (0.5–1 day)
6. **First write tool** — `create_daily_log`; sends and deletes stay off (1 day)
7. **Audit log and limits** — `mcp_audit_log` table, rate limit per user (0.5 day)

8. **Pilot** — add as an org connector in the AI account; test with 2 people for a week (0.5 day setup)

Each step ships to `main` on its own, so the pilot can start after step 3 with read-only tools.

Open Questions

- [ ] ChatGPT Business or Claude Team/Enterprise? Both take remote MCP connectors, but admin setup differs.
- [ ] Sign-in: Supabase Auth's own OAuth server, or a small OAuth layer in the app?
- [ ] Should Bolt move off its prompt-injected knowledge and use `search_knowledge` too?
- [ ] Which roles get `get_exec_summary`: super_admin and admin only?
- [ ] Who owns adding and approving knowledge base documents?